

## 1. What is spring configuration file?

A **Spring configuration file** is a core part of the **Spring Framework** used to define how the various **beans (components/objects)** in a Spring application are created, configured, and wired together (also called **dependency injection**).

**There are Two Main Ways to Configure Spring:**

1. **XML Configuration** (applicationContext.xml)
2. **Java-based Configuration** (@Configuration class)

### 1. XML-Based Configuration (Older Style)

- File: applicationContext.xml or any name like beans.xml
- Located in src/main/resources
- Defines beans using <bean> tags

**Example:**

```
<!-- applicationContext.xml -->
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-
beans.xsd">
    <!-- Defining a bean -->
    <bean id="myService" class="com.example.MyService"/>
</beans>
```

### 2. Java-Based Configuration (Modern Approach)

- Uses @Configuration and @Bean annotation.
- No need for XML.

**Example:**

```
@Configuration
Public class AppConfig {
    @Bean
    Public MyService myService(){
        return new MyService();
    }
}
```

```
AnnotationConfigApplicationContext context = new
AnnotationConfigApplicationContext(AppConfig.class);
MyService service = context.getBean(MyService.class);
```

**Purpose of configuration file.**

- Define Beans.
- Set Beans Scopes like singleton, prototype.

- Configure Dependency Injection.
- Enable Spring module (e.g Spring MVC , Spring Security).
- Externalize properties (e.g Database configs via .properties or .yaml file );

### **In Spring Boot (Modern Spring)**

- Spring Boot minimize the need of configuration files. You typically:
- Use @SpringBootApplication
- Use application.properties or application.yml for external settings
- Spring Boot auto-configures many beans for you.

## **2. What do you ment by IoC container?**

**IoC Container** stands for **Inversion of Control Container**.

It is the **core part of the Spring Framework** that is responsible for:

1. **Creating objects (beans)**
2. **Configuring their dependencies**
3. **Managing their complete life cycle**

### **What is Inversion of Control (IoC)?**

- Normally , we can create objects manually using new keyword.
- In IoC , we give control to spring to create and manage those objects for you.
- This is known as Inversion Of Control : Instead of our code controlling the flow, the framework controls the object creation and wiring.

### **How the IoC Container Works:**

- We define the objects (beans) in a Spring configuration (XML or Java class).
- The Spring IoC Container reads the configuration.
- It instantiates the beans, injects dependencies, and manages their life cycle.

### **Types of IoC Container in Spring:**

- **BeanFactory** : Basic container , lazy Initialization, lightweight.
- **ApplicationContext**: Advanced, eager initialization supports internationalization, events, etc.

Spring container forms the core of the spring framework . The container uses Dependency Injection for managing the application components by creating objects, wiring them together along with configuration and managing their overall life cycles . The instructions for the spring container to do the tasks can be provided either by XML configuration , Java annotations , or Java code.

### 3. What do you understand by Dependency injection?

Dependency injection is a design pattern used in spring (and many other framework ) to remove hard-coded dependencies between objects, making code more flexible, testable and maintainable.

**Definition:** Dependency Injection is a technique where an object receives its dependencies from an external source (like the spring IoC container ), instead of creating them itself.

#### Types Of Dependency Injection in Spring:

- Constructor Injection: Dependencies are passed via constructor.
- Setter Injection: Dependencies are set via setter.
- Field Injection: Dependencies injected directly into field (using @Autowired).

#### Why Use Dependency Injection?

- Loose coupling: We can swap implementation easily.
- Easier testing: We can mock dependencies.
- Better maintainability: Fewer changes needed across classes.
- Framework-managed: Spring handles the wiring for us.

The main idea in Dependency Injection is that you don't have to create your objects but you just have to describe how they should be created.

- The components and services need not to be connected by us in the code directly we have to describe which services are needed by which components in the configuration file. The IoC container present in Spring will wire them up together.
- In Java , the 2 major ways of achieving dependency injection are.
  - Constructor injection: Here, the IoC container invokes the class constructor with a number of arguments where each argument represents a dependency on the other class.
  - Setter injection: Here, the spring container calls the setter methods on the beans after invoking a no-argument static factory method or default constructor to instantiate the bean.

#### Explain the difference between constructor and setter injection?

##### 1 . Partial Injection

##### Constructor injection (Partial Injection Not allowed).

We must have to pass all required dependencies through the constructor.

Example:

```
@Component
public class Student{
    private String name;
```

```

    private int age;

    @Autowired
    Public Student(String name, int age){
        this.name = name;
        this.age = age;
    }
}

```

If spring tries to create this bean, it must provide both name and age, otherwise it fails.

### **Setter Injection(partial Injection Allowed)**

Example:

```

@Component
public class Student{
    private String name;
    private int age;

    @Autowired
    Public setName(String name){
        this.name = name;
    }
    public void setAge(int age){
        this.age = age;
    }
}

```

Only name is injected , age is optional and can be skipped – hence partial injection is possible.

## **2 . Overriding Properties**

### **Constructor Injection (Cannot Be Override)**

```

@Component
public class Car{
    private final String engineType;
    @Autowired
    public Car(String engineType){
        this.engineType = engineType;
    }
    public String getEngineType(){

```

```

        return enginType;
    }
}

```

Setter Injection (Can Be Overridden)

```

@Component
public Class Car{
    private String engineType;

    @Autowired
    public void setEngineType(String engineType){
        this. engineType = engineType;
    }
}

```

Even after the object is created, engineType can be changed by calling the setter again, making it mutable.

### 3. Object Re-Creation

#### Constructor injection (Promotes New Object Creation)

```

Car car1 = new Car("Petrol");
Car car2 = new Car("Diesel");

```

If the engine type changes, we need to create a new Object –No side effects.

#### Setter Injection (No new Object Needed)

```

Car car = new car()
car.setEngineType("Petrol");
car.setEngineType("Diesel");

```

Same Object is used – state change without object recreation.

### 4. Use Case Based on Number of Properties

#### Constructor Injection (Good for many or mandatory dependencies )

```

public class OrderService{
    private final OrderRepository repo;
    private final NotificationService notification;

    @Autowired
    public OrderService(OrderRepository repo , Notificcation
notification){
        this.repo =repo;
        this.notification = notification;
    }
}

```

You inject both services upfront all required, no supprises.

## Setter Injection (Good for few or optional dependencies )

```
public class EmailService{  
    private String smtpServer;  
    public void setSmtpServer(String smtpSever){  
        this.smtpServer = smtpServer;  
    }  
}
```

If smtpServer is optional or comes from a config file later , setter makes sense.

- In constructor injection, partial injection is not allowed whereas it is allowed in setter injection.
- The constructor injection doesn't override the setter property whereas the same is not true for setter injection.
- Constructor injection creates a new instance if any modification is done. The creation of a new instance is not possible in setter injection.
- In case the bean has many properties , then constructor injection is preferred . If it has few properties , then setter injection is preferred.

## What are spring beans?

In Spring, a bean is simply an object that is managed by the Spring IoC container.

**Definition:** A Spring Bean is a java object that is instantiated, assembled and managed by Spring Framework.

## Why they are called "Beans"?

The term comes from JavaBeans – reusable software components .In Spring , a bean is just a POJO registered with the container.

## How Are Beans Created?

Beans can be defined in different ways:

1. Using @Component Annotation

```
@Component  
public class MyService{  
    public void server(){  
        sout("service running...");  
    }  
}
```

2. Using @Bean Method in @Configuration class
- @Configuration

```

public class AppConfig{
    @Bean
    public myService(){
        return new MyService();
    }
}

```

### 3. Using XML Configuration (Old Style)

```
<bean id = "myService" class = "com.example.MyService"/>
```

### Life Cycle Of Spring Beans:

1. Instantiation – Object is created.
2. Dependency Injection – Dependencies are injected.
3. Initialization – Any @PostConstructor or init-method runs
4. Ready to use
5. Destruction – On application shutdown (@predestroy , destroy-method)

### Why Use Beans in Spring?

1. Loose Coupling – Beans are injected, not created manually.
2. Reusability – Beans can be shared across the app.
3. Lifecycle Management – Spring handles initialization and destruction
4. Testability – Beans can be mocked or swapped easily.

In Spring, a **bean** is any object that is **managed by the Spring IoC container**. It is typically defined using annotations like @Component or via configuration classes or XML. Spring takes care of creating, injecting dependencies into, and managing the lifecycle of these beans. This promotes loose coupling, reusability, and easier testing.

- They are the objects forming the backbone of the user's application and are managed by the Spring IoC container.
- Spring beans are instantiated, configured , wired, and managed by IoC container.
- Beans are created with the configuration metadata that users supply to the container (By means of XML or Java annotations configurations.)

### How is the configuration meta data provided to the spring container?

In Spring, configuration metadata tells the IoC container;

What beans to create.

How to inject dependencies

How to manage bean lifecycle,scope,etc.

There are three main ways to provide configuration metadata:

- **XML-Based Configuration (Legacy but still supported)**

You define beans in an XML file (Like applicationContext.xml)

```
<beans xmlns="http://www.springframework.org/schema/beans" ...
>
  <bean id="myService" class="com.example.MyService"/>
  <bean id="myController" class="com.example.MyController">
    <property name="service" ref="myService"/>
  </bean>
</beans>
```

**Spring loads it using:**

**ApplicationContext context = new**

**ClassPathXmlApplicationContext("applicationContext.xml");**

- **Annotation-Based configuration(Modern & common)**

You annotate your classes and spring automatically detects and registers them as beans.

```
@Component
public class MyService{}
@Component
Public class MyController{
    @Autowired
    private MyService myService;
}
```

Enable component scanning using:

```
@Configuration
@ComponentScan(basePackages = "com.example")
Public class AppConfig{}
```

Load with:

```
ApplicationContext context = new
AnnotationConfigApplicationContext(AppConfig.class);
```

- **Java-based configuration:** Spring framework introduced key features as part of new java configuration support. This makes use of the **@Configuration** annotated classes and **@Bean** annotated methods .

@Configuration



```

public class AppConfig{
    @Bean
    public MyService myService(){
        return new MyService();
    }
    @Bean
    public MyController myController(){
        return new MyController(myService());
    }
}

```

Spring loads this config class using:

```

ApplicationContext context = new
AnnotationConfigApplicationContext(AppConfig.class);

```

#### **NOTE THAT**

- **@Bean** annotation has the same role as </bean> element
- Classes annotated with **@Configuration** allow to define inter-bean dependencies by simply calling other @Bean methods in the same class.

#### **Bonus : In Spring Boot**

- Beans are often detected automatically using @ComponentScan.
- External properties (like DB config) are supplied via application.properties or application.yml

#### **What are the bean scopes available in Spring?**

The Spring Framework has five scope supports .They are:

##### **1. Singleton (Default)**

- a. Only One instance per Spring container.
- b. All request for the bean return the same instance

Example:

```

@Component
@Scope("singleton")
Public class MyService{}

```

Use when we want to reuse the same bean across the whole application(e.g. services , DAOs)

##### **2. Prototype**

- a. New instance is created every time the bean is requested from the container.

Example:

```

@Component
@Scope("prototype")
public class TaskProcessor{}

```

Use when we need separate instance for each request (e.g. temporary stateful objects ).

### 3. Request (Web-aware scope)

- a. One instance per HTTP request.
- b. Only valid in web-aware Spring applications(e.g. Spring MVC).

Example:

```
@Component
@Scope("request")
public class RequestScopedBean {}
```

Use when we want to maintain state specific to single HTTP request.

### 4. Session (Web-aware scope)

- a. One instance per user session.

Example:

```
@Component
@Scope("session")
public class SessionScopedBean {}
```

Use when you need to store user-specific data across multiple requests.

### 5. Globale-Session:

- a. One bean per servletContext (i.e. per web application).

Example:

```
@Component
@Scope("application")
public class AppScopedBean {}
```

Use when we need share state across all sessions and requests.

### 6. Websocket

- a. One bean per WebSocket session.

Example:

```
@Component
@Scope("websocket")
public class WebSocketScopedBean {}
```

Use when we are dealing with real-time WebSocket communication.

- **Note:** The last three scope of the bean definition are only if the users use web-aware ApplicationContext container

## **Explain Bean life cycle in Spring Bean Factory Container.**

The bean lifecycle in spring refers to the sequence of steps to bean goes through – from creation to destruction – inside the spring IoC container. When using BeanFactory (the most basic spring container ) , the lifecycle includes fewer features than ApplicationContext, but the core steps remain.

### **1. Instantiation**

- a. Spring creates the bean object using either:
  - i. A constructor (default or parameterized)
  - ii. A factory method

```
public class MyBean {  
    public MyBean() {  
        System.out.println("1. Bean instantiated"); } }  
}
```

### **2. Populate Properties (Dependency Injection)**

- a. Spring injects dependencies (via constructor,setter or fields)

```
public void setDependency(MyDependency dep) {  
    this.dep = dep;  
    System.out.println("2. Dependencies injected");  
}
```

### **3. BeanNameAware (Optional)**

- a. If our bean implements BeanNameAware , spring calls:

```
setBeanName(String name);
```

### **4. BeanFactoryAware (optional)**

- a. If bean implements BeanFactoryAware, Spring passes the BeanFactory to:

```
setBeanFactory(BeaFactory factory);
```

### **5. InitializingBean / afterPropertiesSet() (Optional)**

- a. If bean implements InitializingBean, Spring calls:

```
afterPropertiesSet();
```

### **6. Custom init-method (if defined)**

- a. If defined in XML or Config, Spring calls the custom initialization method.

```
<bean id="myBean" class="com.MyBean" init-method="init"/>
```

**OR**

```

@PostConstruct
public void init() {
    System.out.println("6. Custom init method");
}

```

### 7. Bean is Ready to Use

- a. At this stage, the bean is fully initialized and managed by the container.

### 8. DisposableBean / destroy() (optional)

- a. If bean implements DisposableBean, Spring calls: Destroy();

### 9. Custom destroy-method(if needed)

- a. If Configured:  
`<bean id="myBean" class="com.MyBean" destroy-method="cleanup"/>`

OR With @Predestroy:

```

@PreDestroy
public void cleanup() {
    System.out.println("9. Cleanup called");
}

```

## What is autowiring and name the different modes of it?

The IoC container autowires relationships between the application beans. Spring lets collaborators resolve which bean has to be wired automatically by inspecting the contents of the BeanFactory.

### Why Use Autowiring ?

1. Reduce boilerplate XML code or manual setter calls.
2. Promotes loose coupling
3. Helps manage complex dependency graphs in large applications.

### Different modes of this process are:

- **no:** This means no autowiring and is default setting. An explicit bean reference should be used for wiring.
- **byName:** The bean dependency is injected according to the **name of the bean**. This matches and wires its properties with the beans defined by the same names as per configuration.
- **byType:** This injects the bean dependency based on type.
- **constructor:** Here, it injects the bean dependency by calling the constructor of the class. It has a large number of parameters.
- **Autodetect:** First the container tries to wire using autowire by the constructor, if it isn't possible then it tries to autowire by byType.

## What are the limitations of autowiring?

- Overriding possibility : Dependencies are specified using <constructor-arg> and <property> settings that override autowiring. This means: Even if autowiring is enable, manual configuration takes precedence , which can lead to confusion if both are used together.

Best approach: Stick to one approach (autowiring or explicit config ) to avoid conflicts.

- Data types restrictions: Primitive data types, String and Classes can't be autowired.

Spring can't autowire:

- int, Boolean,float,etc.
- String
- Non\_spring-managed classes.

Solution: Use @Value for primitive or string injection:

```
@Value
private String appName;
```

Or define a bean for complex types:

```
@Bean
public MyCustomClass cutomeClass(){
    return new MYCustomClass();
}
```

## Additional Key Limitations

- Multiple bean of same type
  - When more than one bean of the same type exists, Spring cannot resolve which one to inject.
  - Use @Qualifer to avoid ambiguity
- No autowiring for static fields or Local Variables
  - Spring cannot inject into static fields or local method variables
- Hard to identify Dependencies
  - Autowiring hides dependencies, making the code less readable and harder to debug in large systems.

## **What do you understand by the term “Spring Boot”?**

Spring Boot is an open-source, java-based framework that provides support for rapid application development and gives a platform for developing stand-alone and production-ready spring application with a need for very few configuration.

**Spring Boot** is a framework built on top of the Spring Framework that makes it **easier and faster to develop Spring-based applications** by reducing boilerplate configuration.

Key features of spring boot

1. Auto- configuration: Automatically configures beans based on classpath contents and project settings (e.g. adds a DataSource bean if JDBC is on classpath).
2. Standalone Applications: Create applications with an embedded server like tomcat or jetty – no need to deploy WAR files separately.
3. Opinioned Defaults: Provides sensible defaults to reduce configuration time, yet fully customizable
4. Production-ready feature: Built-in features like health checks, metrics, and monitoring via spring boot actuator.
5. Spring boot starter dependencies : Comes with pre-defined starter packages like
  - a. spring-boot-starter-web
  - b. spring –boot-starter-data-jpa
  - c. spring-boot-starter-security
6. No XML configuration required: Uses java based and annotation based configuration, improving readability and maintainability.

**Spring Boot is a rapid development framework that simplifies Spring application development through auto-configuration, embedded servers, and minimal setup.**

## **Explain the advantages of using spring boot for application development.**

- Spring Boot helps to create stand-alone application which can be started using java.jar (Doesn't require configuration WAR files).
- Spring Boot also offers pinpointed 'started' POMs to Maven configuration.
- Has provision to embed Undertow , Tomcat , Jetty, or other web servers directly.
- Auto- configuration: Provides a way to automatically configure an application based on the dependencies present on the classpath.
- Spring Boot was developed with the intention of lessening the lines of code.

- If offers production-ready support like monitoring and apps developed using spring boot are easier to launch.

### **Differentiate between Spring and Spring Boot.**

- The Spring Framework provides multiple features like dependency injection, data binding , aspect-orientated programming (AOP) , data access, and many more that help easier development of web application whereas spring boot helps in easier usage of the spring framework by simplifying or managing various loosely coupled blocks which are tedious and have a potential of becoming messy.
- Spring boot simplifies commonly used spring dependencies and runs application straight from a command line. It also doesn't require an application container and it helps in monitoring several components and configures them externally.

What Does @SpringBootApplication Do internally?

The @SpringBootApplication annotation is a convenience annotation that combine three key Spring annotations:

@Target(ElementType.TYPE)

@Retention(RetentionPolicy.RUNTIME)

@SpringBootConfiguration

@EnableAutoConfiguration

@ComponentScan

```
public @interface SpringBootApplication {
}
```

Internal Breakdown:

1. @SpringBootConfiguration
  - a. It is a special form of @Configuration
  - b. Marks the class as configuration class for spring
  - c. Used to define beans using @Bean methods
2. @EnableAutoConfiguration
  - a. Tells spring boot to auto-configure the application based on dependencies available in the classpath.
  - b. For example: if spring-boot-starter-web is present , it configures Tomcat, Jackson, MVC , etc.
3. @ComponentScan
  - a. Enables component scanning in the packages where the application class is located and its sub-packages.
  - b. Automatically detects and registers beans like @Component, @Repository and @Controller.

## What is Spring Boot Dependency Management System?

Spring Boot uses a built-in dependency management system that helps you manage versions of commonly used libraries automatically, so you don't have to specify versions manually.

Key Concepts:

### 1. Spring Boot parent POM

- a. When you use Spring Boot, you typically declare this in your pom.xml

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>3.2.0</version> <!-- Example version -->
</parent>
```

- b. This parent POM provides:

- i. Default plugin configurations
- ii. Java version setup
- iii. Dependency versions for spring boot and third party libraries.

### 2. Managed Dependency versions: We don't need to specify versions for commonly used dependencies (e.g. Spring, Hibernate, Jackson).

Example:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

No need to write a version tag – Spring Boot manages it internally.

### 3. Spring Boot Starters: starters are predefined dependencies bundles for common use cases (e.g. web, jpa, security )

Example:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```

Automatically brings in Hibernate, Spring Data, and required versions.



4. Customizing version (if needed): If we want to override versions, we can still do it explicitly.

Example:

```
<properties>  
  <jackson.version>2.15.0</jackson.version>  
</properties>
```

Spring Boot's dependency management system, powered by its parent POM, handles library versions automatically and uses starter dependencies to simplify configuration and development.

What are the Possible Sources of External Configuration in spring boot?  
Spring boot allows externalizing configuration so we can run the same code in different environment (dev, test, production) without changing the code.

Common External Configuration Sources (in order to precedence):

1. Command-line arguments

Example: --server.port==8080

2. Java System properties

Example: -Dserver.port=9090

3. Environment variables

Example: SERVER\_PORT=9090

4. application.properties or application.yml files

- a. located in: src/main/resources
- b. External location (e.g. /config/)
- c. Current working directory

5. Profile-specific properties files

Example : application-dev.properties , application-prod.yml

Used with spring.profiles.active=dev

6. Spring @PropertySource annotations

Load properties from custom files.

@PropertySource("classpath:myconfig.properties")

7. Config data from config directories

Automatically loaded from config/ folders next to your JAR or in classpath:/config/.

8. Spring cloud config server(if used)

Fetches remote configuration from Git, Vault or other sources.

9. Default values in your code

Example:

```
@Value("${app.name:MyApp}")
```

Spring Boot supports multiple external configuration sources such as properties files, environment variables, system properties, and command-line args — enabling flexibility and environment-based customization.

Can We Change the Default Port of the Embedded Tomcat Server in Spring Boot?

Yes, we can change the default port (which is 8080) of the embedded Tomcat server in Spring Boot.

**Ways to Change the Port:**

1. **Using** application.properties : server.port=9090

2. **Using** application.yml :

```
server:  
  port: 9090
```

3. **Via Command Line:** java -jar app.jar --server.port=9090

4. **Using Java System Property:** -Dserver.port=9090

Programmatically (Not recommended unless needed)

@Bean

```
public WebServerFactoryCustomizer<ConfigurableWebServerFactory>  
customizer() {  
    return factory -> factory.setPort(9090);  
}
```

Yes, Spring Boot allows changing the embedded server port using properties, YAML, or programmatically — the most common way is setting server.port in application.properties.

## Can you tell how to exclude any package without using the basePackages filter?

Yes! You can exclude a package from component scanning **without using** basePackages by using the @ComponentScan **annotation with the** excludeFilters **attribute**.

### How to Exclude a Package Without Using basePackages

You use the excludeFilters option of @ComponentScan to specify what to **exclude** — based on annotations, assignable types, or regex patterns.

Example: Exclude a Package (e.g., com.example.unwanted)

```
@SpringBootApplication
@ComponentScan(
    excludeFilters = @ComponentScan.Filter(
        type = FilterType.REGEX,
        pattern = "com\\.example\\.\\.unwanted\\.\\.*)"
    )
)
public class MyApp {
    public static void main(String[] args) {
        SpringApplication.run(MyApp.class, args);
    }
}
```

Filter Types You Can Use:

| Filter Type     | Use Case Example                          |
|-----------------|-------------------------------------------|
| ANNOTATION      | Exclude by annotation (e.g., @Controller) |
| ASSIGNABLE_TYPE | Exclude specific class types              |
| REGEX           | Exclude packages or class names via regex |
| CUSTOM          | Use your own TypeFilter implementation    |

To exclude a package from scanning without using basePackages, use @ComponentScan with excludeFilters and set FilterType to REGEX or others as per need.

## How to Disable a Specific Auto-Configuration Class in Spring Boot?

You can **disable specific auto-configuration classes** using the `exclude` or `excludeName` attribute of the `@SpringBootApplication` or `@EnableAutoConfiguration` annotation.

### 1. Using `exclude`

```
@SpringBootApplication(exclude =
    {DataSourceAutoConfiguration.class})
public class MyApplication {
    public static void main(String[] args) {
        SpringApplication.run(MyApplication.class, args);
    }
}
```

This disables the `DataSourceAutoConfiguration` class.

### 2. Using `excludeName` (String-based class name)

Useful if the class is not on the classpath during compile time.

```
@SpringBootApplication(
    excludeName =
    {"org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfi
    guration"})
)
```

### 3. Using `spring.autoconfigure.exclude` in `application.properties`

```
spring.autoconfigure.exclude=org.springframework.boot.autoconfigure.jdbc.Data
SourceAutoConfiguration
```

This is helpful when you want to avoid changes in Java code.

When Would You Use This?

- You want full control over configuration (e.g., define your own `DataSource`, `Security`, or `JPA` setup).
- You want to reduce startup time or avoid conflicts with custom beans.

You can disable specific auto-configuration classes using the `exclude` or `excludeName` attribute of `@SpringBootApplication` or through `application.properties`.

Can the default web server in the Spring Boot application be disabled?

Yes, the **default web server in a Spring Boot application can be disabled**, and this is a commonly asked interview question — especially for **non-web** or **CLI-based Spring Boot applications**.

Why Disable the Web Server?

To build a **non-web** application like:

Command-line tools

Background jobs

Microservices that communicate over messaging (like Kafka, RabbitMQ)

**How to Disable the Embedded Web Serve**

### 1. Set Application Type to NONE (Recommended)

In the main method:

```
public static void main(String[] args) {  
    SpringApplication app = new SpringApplication(MyApp.class);  
    app.setWebApplicationType(WebApplicationType.NONE);  
    app.run(args);  
}
```

This disables the web server completely. Suitable for CLI apps or background services.

### 2. Using application.properties (Less common)

```
spring.main.web-application-type=none
```

### 3. Exclude Web Server Auto-Configuration

```
@SpringBootApplication(  
    exclude = {EmbeddedWebServerFactoryAutoConfiguration.class}  
)
```

This method is error-prone and not recommended unless you have very specific needs.

Yes, Spring Boot allows disabling the embedded web server by setting the application type to `WebApplicationType.NONE`. This is useful for CLI apps or backend services that don't serve HTTP requests.

What are the uses of @RequestMapping and @RestController annotations in Spring Boot?

@RestController

**Purpose:** Marks the class as a **RESTful controller** where **each method returns a response body**, not a view.

**Combines:**

@Controller + @ResponseBody

@RestController

```
public class MyController {
```

```
    @GetMapping("/hello")
    public String sayHello() {
        return "Hello, World!";
    }
}
```

No need to write @ResponseBody on each method.

@RequestMapping

**Purpose:** Maps **HTTP requests** to handler methods in controller classes.

**Can be applied:**

On class level → base URI

On method level → specific endpoint or HTTP method

@RestController

@RequestMapping("/api")

```
public class ProductController {
```

```
    @RequestMapping(value = "/products", method = RequestMethod.GET)
    public List<String> getProducts() {
        return List.of("Pen", "Notebook");
    }
}
```

This maps GET /api/products to the getProducts() method.

@RestController is used to build REST APIs by combining @Controller and @ResponseBody, while @RequestMapping is used to map URLs and HTTP methods to specific controller methods.

## 27. What is Spring AOP?

Spring AOP (Aspect-Oriented Programming) is a programming paradigm used in Spring to modularize cross-cutting concerns like logging, transaction management, and security.

Key Points:

AOP separates cross-cutting logic from business logic.

Spring AOP is **proxy-based** and works only for method execution join points.

Internally uses **JDK dynamic proxies** or **CGLIB proxies**.

Real-world Use Cases:

Logging every method call

Security checks

Measuring performance (execution time)

Transaction management

Example:

1. Add Dependency (Maven):

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-aop</artifactId>
</dependency>
```

2. Create Aspect:

@Aspect

@Component

```
public class LoggingAspect {
```

```
    @Before("execution(* com.example.service.*(..))")
    public void logBefore(JoinPoint joinPoint) {
        System.out.println("Method called: " +
joinPoint.getSignature().getName());
    }
}
```

3. Target Business Logic:

@Service

```
public class ProductService {
    public void getAllProducts() {
        System.out.println("Fetching products...");
    }
}
```

Output:

Method called: getAllProducts

Fetching products...

What is an Advice? Explain its Types in Spring.

**Advice:** It is the **action taken** by an aspect at a particular join point.

| Type           | Description                                             | Example Annotation |
|----------------|---------------------------------------------------------|--------------------|
| Before         | Runs before the method execution                        | @Before            |
| After          | Runs after method completes (success or failure)        | @After             |
| AfterReturning | Runs only after successful method execution             | @AfterReturning    |
| AfterThrowing  | Runs if method throws an exception                      | @AfterThrowing     |
| Around         | Runs before and after, and can control method execution | @Around            |

Example:

@Aspect

@Component

```
public class LoggingAspect {
```

```
    @AfterReturning("execution(*
com.example.service.ProductService.*(..))")
    public void afterSuccess() {
        System.out.println("Method executed successfully.");
    }
```

```
    @AfterThrowing("execution(*
com.example.service.ProductService.*(..))")
    public void afterException() {
        System.out.println("Exception occurred during method execution.");
    }
}
```

29. What is Spring AOP Proxy Pattern?

Spring AOP uses the Proxy Design Pattern to implement AOP features:

Types:

JDK Dynamic Proxy – If the target object implements interfaces.

CGLIB Proxy – If the target object does not implement interfaces.

Example:

```
ProductService proxy = (ProductService) Proxy.newProxyInstance(
    ProductService.class.getClassLoader(),
    new Class[]{ProductService.class},
    new MyInvocationHandler(new ProductServiceImpl())
);
```

Spring hides all this internally, but the mechanism is the same.



## Spring JDBC API Classes

| Class                      | Purpose                                |
|----------------------------|----------------------------------------|
| JdbcTemplate               | Core class for JDBC operations         |
| NamedParameterJdbcTemplate | Use named parameters in SQL            |
| SimpleJdbcInsert           | Simplified insert operations           |
| SimpleJdbcCall             | Simplified stored procedure call       |
| RowMapper                  | Maps rows of ResultSet to Java objects |

### Fetching Records Using JdbcTemplate

Fetch List:

@Autowired

```
JdbcTemplate jdbcTemplate;  
public List<User> getUsers() {  
    String sql = "SELECT * FROM users";  
    return jdbcTemplate.query(sql, new  
        BeanPropertyRowMapper<>(User.class));  
}
```

Fetch Single Object:

```
public User getUserById(int id) {  
    String sql = "SELECT * FROM users WHERE id = ?";  
    return jdbcTemplate.queryForObject(sql, new  
        BeanPropertyRowMapper<>(User.class), id);  
}
```

What is Hibernate ORM Framework?

**Hibernate** is an Object-Relational Mapping (ORM) framework that:

Maps Java objects to relational database tables

Handles automatic CRUD operations

Supports caching, lazy loading, and relationships

Example:

@Entity

```
public class User {  
    @Id @GeneratedValue  
    private int id;  
    private String name;  
}
```

## Two Ways to Access Hibernate with Spring

### 1. Using HibernateTemplate (older)

```
hibernateTemplate.save(user);
```

### 2. Using SessionFactory + @Transactional (recommended)

@Service

```
public class UserService {
```

```
    @Autowired
```

```
    private SessionFactory sessionFactory;
```

```
    @Transactional
```

```
    public void saveUser(User user) {
```

```
        sessionFactory.getCurrentSession().save(user);
```

```
    }
```

```
}
```

## What is Hibernate Validator Framework?

Hibernate Validator is the reference implementation of **JSR-380 (Bean Validation 2.0)**. It validates JavaBeans by using annotations.

Common Annotations:

@NotNull

@Size(min=2, max=30)

@Email

@Pattern

Example:

```
public class User {
```

```
    @NotNull
```

```
    private String name;
```

```
    @Email
```

```
    private String email;
```

```
}
```

Used in Spring REST controller:

```
@PostMapping("/users")
```

```
public ResponseEntity<?> createUser(@Valid @RequestBody User user) {
```

```
    // Spring handles validation errors
```

```
}
```

What is HibernateTemplate?

HibernateTemplate is a **helper class provided by Spring** to simplify Hibernate data access.

Responsibilities:

Automatically handles Session lifecycle

Catches Hibernate exceptions and rethrows as Spring's  
DataAccessException

Example:

@Autowired

private HibernateTemplate hibernateTemplate;

```
public void save(User user) {  
    hibernateTemplate.save(user);  
}
```

**Note:** Deprecated in favor of modern SessionFactory + @Transactional usage.

36. What is the Spring MVC Framework?

Spring MVC is a module in Spring for building web applications using the Model-View-Controller (MVC) design pattern.

It separates business logic (Model), UI (View), and input logic (Controller).

Built on top of Servlet API.

Highly configurable and integrates well with Spring features like DI, AOP, and more.

37. Benefits of Spring MVC Over Other MVC Frameworks

Loose coupling using Dependency Injection

Annotation-driven configuration (@Controller, @RequestMapping)

Integration with Spring features like AOP, Security, Transactions

Testability: Controllers are POJOs

Flexible View Resolvers (JSP, Thymeleaf, PDF, Excel, etc.)

38. What is DispatcherServlet in Spring MVC?

The DispatcherServlet is the front controller in Spring MVC that:

Receives all HTTP requests

Delegates to appropriate controllers

Selects the View using ViewResolver

Sends the response to the client

Defined in web.xml or auto-registered in Spring Boot.

39. What is View Resolver Pattern in Spring MVC?

ViewResolver maps a view name returned by a controller to an actual View (e.g., JSP).

Example:

@Bean

```
public InternalResourceViewResolver viewResolver() {  
    InternalResourceViewResolver resolver = new  
InternalResourceViewResolver();  
    resolver.setPrefix("/WEB-INF/views/");  
    resolver.setSuffix(".jsp");  
    return resolver;  
}
```

40. What is the @Controller Annotation Used For?

@Controller marks a class as a Spring MVC Controller that can handle web requests.

@Controller

```
public class HomeController {  
    @GetMapping("/home")  
    public String home() {  
        return "index";  
    }  
}
```

41. Can You Create a Controller Without Using @Controller or

@RestController?

Yes, but it's not recommended. You can use XML configuration to register a bean and handler mappings, but annotations provide simplicity and clarity.

42. What is ContextLoaderListener and What Does It Do?

ContextLoaderListener is a servlet listener that loads the Spring root application context (services, DAOs) during web application startup.

43. @RequestParam vs @PathVariable

| Aspect | @RequestParam       | @PathVariable       |
|--------|---------------------|---------------------|
| Source | Query Parameter     | URI Path Segment    |
| Syntax | /user?id=1          | /user/1             |
| Usage  | @RequestParam("id") | @PathVariable("id") |

#### 44. What is the Model in Spring MVC?

Model is used to pass data from the controller to the view.

```
model.addAttribute("user", user);
```

In the view (JSP or Thymeleaf), the data can be accessed for rendering.

#### 45. What is the Use of @Autowired Annotation?

@Autowired injects dependencies automatically by type. Can be applied to fields, constructors, or setters.

```
@Autowired
```

```
private UserService userService;
```

#### 46. Role of @ModelAttribute Annotation

@ModelAttribute binds form data or request parameters to model objects.

```
@PostMapping("/save")
```

```
public String save(@ModelAttribute("user") User user) {
```

```
    // user object populated
```

```
}
```

#### 47. Importance of web.xml in Spring MVC

In traditional Spring web apps:

Declares DispatcherServlet

Maps servlet to URL pattern

Declares ContextLoaderListener

Not required in Spring Boot (replaced by annotations/config).

#### 48. Types of Spring MVC Dependency Injection

Constructor-based DI

Setter-based DI

Field-based DI (using @Autowired directly on fields)

#### 49. Importance of Session Scope

Beans remain active for the entire session duration.

Useful for storing user-specific data like login details, cart info.

```
@Scope("session")
```

```
@Component
```

```
public class Cart {}
```

## 50. Importance of @Required Annotation

@Required ensures a bean property is set. If not, container throws an exception.

@Required

```
public void setName(String name) {  
    this.name = name;  
}
```

Deprecated in modern Spring in favor of @Autowired with required=true.

## 51. @Autowired vs @Inject

|                  |                     |                         |
|------------------|---------------------|-------------------------|
| Feature          | @Autowired (Spring) | @Inject (Javax/Jakarta) |
| Provided By      | Spring Framework    | Java Standard (JSR-330) |
| Required Config? | No                  | Yes (javax.inject lib)  |
| Extra Features   | required attribute  | No                      |

## 52. Are Singleton Beans Thread-safe?

No. Singleton beans are not thread-safe by default.

You must handle synchronization if beans are shared between threads.

## 53. How to Achieve Thread-safety in Beans?

Avoid shared mutable state

Use @Scope("prototype") for per-request beans

Synchronize critical sections

Use ThreadLocal if needed

## 54. Significance of @Repository Annotation

@Repository marks a class as a DAO (data access object).

Enables exception translation to Spring's DataAccessException

Makes class a Spring-managed bean

@Repository

```
public class UserDao {}
```

## 55. How is DispatcherServlet Instantiated?

In XML config: declared in web.xml

In Spring Boot: auto-configured and mapped to /

Spring Boot registers it via DispatcherServletAutoConfiguration

@Bean

```
public DispatcherServlet dispatcherServlet() {  
    return new DispatcherServlet();  
}
```

56. How is the root application context in Spring MVC loaded?

**Answer:**

In a Spring MVC application, the **root application context** is loaded by a **ContextLoaderListener** that's configured in web.xml or via Java config if using a Servlet 3.0+ container.

The root context is responsible for shared, **non-web-specific beans** (like Services, Repositories, Security config).

It is **initialized before** the DispatcherServlet.

In web.xml:

xml

CopyEdit

```
<listener>
```

```
  <listener-
```

```
class>org.springframework.web.context.ContextLoaderListener</listener-  
class>
```

```
</listener>
```

```
<context-param>
```

```
  <param-name>contextConfigLocation</param-name>
```

```
  <param-value>/WEB-INF/applicationContext.xml</param-value>
```

```
</context-param>
```

Interview Tip:

Mention that the **DispatcherServlet** also loads its own **child**

**WebApplicationContext**, usually configured in dispatcher-servlet.xml.

Root context = global/shared beans. Dispatcher context = web-related beans (Controllers, ViewResolvers).

57. How does the Spring MVC flow look like?

**Answer:**

Spring MVC follows the **Front Controller Pattern** via DispatcherServlet.

Flow:

**Request** hits DispatcherServlet.

**HandlerMapping** determines the appropriate Controller based on the URL.

**HandlerAdapter** calls the mapped Controller method.

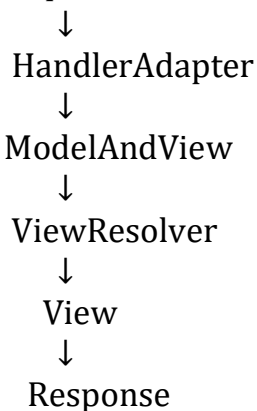
Controller processes logic, returns a **ModelAndView** object.

**ViewResolver** resolves the view name to a physical view.

Dispatcher renders the view and sends the **response**.

Diagram:

Client → DispatcherServlet → HandlerMapping → Controller



Interview Tip:

Explain the role of @RequestMapping, @Controller, and ViewResolvers (like InternalResourceViewResolver).

58. Where does the access to the model from the view come from?

**Answer:**

Spring uses the **Model** (or ModelMap, ModelAndView) to pass data from controller to view.

This model is **internally exposed to the view** by the DispatcherServlet.

Views (like JSPs) access model attributes using \${attributeName} (via JSTL or Spring tags).

Example:

```
java
CopyEdit
@GetMapping("/greet")
public String greet(Model model) {
    model.addAttribute("name", "Siddhesh");
    return "hello"; // hello.jsp
}
```



In hello.jsp:

jsp

CopyEdit

```
<p>Hello, ${name}</p>
```

59. Why do we need BindingResults?

**Answer:**

BindingResult is used to capture **validation errors** after form data is bound to a model object.

It **must come immediately after** the validated model attribute in the method parameter list.

Contains field errors, global errors, and allows custom error handling.

Example:

java

CopyEdit

```
@PostMapping("/register")
```

```
public String register(@Valid User user, BindingResult result) {
```

```
    if (result.hasErrors()) {
```

```
        return "registerForm";
```

```
    }
```

```
    return "success";
```

```
}
```

Interview Tip:

Explain annotations like @Valid, @NotNull, and how they integrate with BindingResult.

60. What are Spring Interceptors?

**Answer:**

Interceptors in Spring MVC are similar to **Servlet Filters** but are **Spring-specific**.

Implemented using HandlerInterceptor interface.

Used for **logging, authentication, timing, or modifying requests/responses**.

Methods:

preHandle() – before controller method.

postHandle() – after controller but before view rendering.

afterCompletion() – after view rendering.

Example:

java

CopyEdit

```
public class MyInterceptor implements HandlerInterceptor {
    public boolean preHandle(...) {
        // logic before controller
        return true;
    }
}

```

Registered in WebMvcConfigurer.

61. Is there any need to keep spring-mvc.jar on the classpath or is it part of spring-core?

**Answer:**

No, spring-webmvc.jar is **separate** and **not part of** spring-core.

You must include it explicitly in your build tool (Maven/Gradle).

It contains Spring MVC components like DispatcherServlet, Controller, etc.

Maven:

xml

CopyEdit

```
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-webmvc</artifactId>
    <version>5.3.30</version>
</dependency>

```

62. What are the differences between the <form:form> vs <form> tags?

**Answer:**

<form:form> – Spring's **tag library** that binds forms to model objects.

<form> – standard HTML tag.

<form:form> features:

Auto-binding to model.

Validation message support.

Works with @ModelAttribute.

Example:

jsp

CopyEdit

```
<form:form modelAttribute="user" method="post" action="/save">
    <form:input path="username"/>
</form:form>

```

63. How is form data validation done in Spring Web MVC?

**Answer:**

Spring uses **JSR-380 (Bean Validation API)** with @Valid or @Validated.

Steps:

Add validation annotations to model.

Annotate controller param with @Valid.

Use BindingResult to handle errors.

Model:

```
java
CopyEdit
public class User {
    @NotBlank
    private String name;
}
```

Controller:

```
java
CopyEdit
@PostMapping("/submit")
public String submit(@Valid User user, BindingResult result) {
    if (result.hasErrors()) return "form";
    return "success";
}
```

64. How to get ServletConfig and ServletContext in a Spring bean?

**Answer:**

You can inject ServletContext and ServletConfig using:

Option 1: Implement ServletContextAware:

```
java
CopyEdit
@Component
public class MyBean implements ServletContextAware {
    public void setServletContext(ServletContext servletContext) {
        // use it
    }
}
```

Option 2: Autowire it:

```
java
CopyEdit
@Autowired
private ServletContext servletContext;
```

65. Differentiate between a BeanFactory and an ApplicationContext.

| Feature              | BeanFactory                 | ApplicationContext                |
|----------------------|-----------------------------|-----------------------------------|
| Lazy Initialization  | Yes                         | No (Eager by default)             |
| Internationalization | No                          | Yes                               |
| Event Propagation    | No                          | Yes                               |
| BeanPostProcessors   | Manual                      | Automatic                         |
| Use Case             | Lightweight apps, IoT, etc. | Web/Enterprise-level applications |

66. How are i18n and localization supported in Spring MVC?

**Answer:**

Spring supports i18n using ResourceBundleMessageSource and locale resolver.

Steps:

Define messages.properties, messages\_fr.properties, etc.

Configure MessageSource bean.

Use LocaleChangeInterceptor to switch language.

In Controller/View:

jsp

CopyEdit

```
<spring:message code="welcome.message"/>
```

67. What do you understand by MultipartResolver?

**Answer:**

MultipartResolver handles file uploads in Spring MVC.

Steps:

Add commons-fileupload or use Servlet 3.0

StandardServletMultipartResolver.

Configure in config file.

Bean Config:

java

CopyEdit

@Bean

```
public MultipartResolver multipartResolver() {  
    return new StandardServletMultipartResolver();  
}
```

68. How is it possible to use the Tomcat JNDI DataSource in the Spring applications?

**Answer:**

You can define the JNDI datasource in Tomcat's context.xml:

xml

CopyEdit

```
<Resource name="jdbc/MyDS" auth="Container"
type="javax.sql.DataSource"
... />
```

Then, in Spring:

java

CopyEdit

@Bean

```
public DataSource dataSource() throws NamingException {
    JndiTemplate jndi = new JndiTemplate();
    return (DataSource) jndi.lookup("java:comp/env/jdbc/MyDS");
}
```

69. What will be the selection state of a checkbox input if the user first checks the checkbox and gets validation errors in other fields and then unchecks the checkbox?

**Answer:**

If the checkbox is **unchecked**, no value is submitted, and Spring treats it as false.

If the form is redisplayed after validation error, the **model value is reused**.

If model was updated to false, the checkbox will remain unchecked.

Tip:

Ensure the checkbox is bound to a boolean field in the model:

jsp

CopyEdit

```
<form:checkbox path="agreeToTerms"/>
```